

# Aplicação Kmeans

## Projeto Concorrente I

---

Rary Emanuel Gonçalves Coringa

DIM0124 - Programação Concorrente

Departamento de Informática e Matemática Aplicada

Universidade Federal do Rio Grande do Norte

Introdução

Implementação Serial

Implementação Concorrente (v2)

Implementação Otimizada (v3)

Resultados e Lições Aprendidas

# Introdução

---

# Algoritmo K-Means

Algoritmo de agrupamento não supervisionado que particiona  $N$  pontos em  $k$  grupos.

**Duas fases por iteração:**

1. **Atribuição** — cada ponto é atribuído ao centróide mais próximo (distância Euclidiana)
2. **Atualização** — cada centróide é recomputado como a média dos seus pontos atribuídos

Repete até os centróides estabilizarem (deslocamento máximo  $\leq$  tolerância) ou `maxIter` ser atingido.

**Complexidade por iteração:**  $O(N \cdot k \cdot d)$

- $N$  = número de pontos     $k$  = clusters     $d$  = dimensões

# Dataset e Configuração

## Dataset

- $\approx 5,75$  milhões de linhas
- 38 atributos numéricos
- $\approx 1,75$  GB em memória
- Pedidos de e-commerce (sintético)

## Parâmetros de agrupamento

- $k = 10$  clusters
- $\text{maxIter} = 20$  (execução completa)
- $\text{tolerance} = 10^{-6}$
- JDK 25.0.1, 16 CPUs lógicas

| order_id   | latitude | longitude | seller_rating | shipping_fee | items_count | delivery_hours |
|------------|----------|-----------|---------------|--------------|-------------|----------------|
| O-00000001 | 54.8877  | -1.6479   | 4.46          | 0.60         | 4           | 53             |
| O-00000002 | 51.4400  | -0.0266   | 4.30          | 9.65         | 8           | 58             |
| O-00000003 | 51.4070  | -2.4147   | 4.52          | 5.94         | 5           | 122            |

Linhas de amostra (4 colunas de ID excluídas dos atributos; 38 colunas numéricas utilizadas)

# Estratégia de Benchmarking

| Ferramenta | Finalidade                    | Config                       |
|------------|-------------------------------|------------------------------|
| JCStress   | Detecção de race condition    | 6 forks, $5 \times 1s$       |
| JMH        | Microbenchmark (por operação) | 1 fork, 1wu + $3 \times 10s$ |
| JMeter     | Macrobenchmark (throughput)   | 120s + 30s ramp              |
| JFR        | Perfil de CPU e GC            | settings=profile             |

- JMH mede custo de computação isolado (B1–B4)
- JMeter mede throughput do sistema sob carga concorrente
- JFR revela onde o tempo é efetivamente gasto

# Implementação Serial

---

Uma única thread trata tudo sequencialmente:

1. Lê uma linha do CSV
2. Converte doubles (`FloatingDecimal.parseDouble`)
3. Encontra o centróide mais próximo (loop `squaredDistance`)
4. Acumula somas e contagens em arrays compartilhados
5. Repete para todos os 5,75 milhões de pontos
6. Recomputa centróides, verifica convergência

Nenhuma sincronização necessária — uma thread, sem estado compartilhado.



| ID | Benchmark                                          | Score              | Unidade |
|----|----------------------------------------------------|--------------------|---------|
| B1 | Execução completa ( $k=10$ , $\text{maxIter}=20$ ) | $212.57 \pm 34.48$ | s       |
| B2 | Por iteração ( $k=10$ , 5 iter)                    | $60.05 \pm 2.46$   | s       |
| B3 | closestCentroid                                    | 96.82              | ns      |
| B4 | squaredDistance                                    | 9.31               | ns      |

Cada iteração leva  $\approx 60$  s — maior parte aguardando o disco e fazendo parsing.

| Método                                   | CPU % | Categoria      |
|------------------------------------------|-------|----------------|
| <code>FloatingDecimal.parseDouble</code> | ≈87%  | parsing de CSV |
| <code>String.split</code>                | —     | parsing de CSV |
| <code>BufferedReader.readLine</code>     | —     | I/O            |
| <code>KMeans.closestCentroid</code>      | ≈13%  | Algoritmo      |
| <code>KMeans.squaredDistance</code>      | —     | Algoritmo      |

87% do tempo é gasto no parsing de CSV, não na computação.

Paralelizar os 13% sem tratar os 87% não vai ajudar.

## Implementação Concorrente (v2)

---

**Padrão batch-fork:** a thread principal lê o CSV sequencialmente e divide os pontos em lotes de 5.000, submetendo cada um a um pool de workers.

- Cada worker acumula em arrays **locais** (`localSums`, `localCounts`) — sem estado compartilhado
- Retorna um record `PartialResult`
- Thread principal reduz todos os parciais após coletar os `Futures`

**Três variantes exploradas:**

**v2-platform** Pool fixo de  $N$  threads platform (escalonadas pelo kernel)

**v2-virtual** Pool fixo de  $N$  threads virtual (escalonadas pela JVM)

**v2-hybrid**  $N$  workers platform + 1 coordenador virtual

## v2 — Race Conditions (JCStress)

Quatro regiões críticas identificadas — todas relacionadas à acumulação compartilhada.

| ID   | Operação                          | Resultado | Freq. máx |
|------|-----------------------------------|-----------|-----------|
| JCS1 | closestCentroid (somente leitura) | ✓ PASSOU  | —         |
| JCS2 | squaredDistance (somente leitura) | ✓ PASSOU  | —         |
| JCS3 | counts[c]++ (compartilhado)       | ✗ FALHOU  | 28%       |
| JCS4 | sums[c][d] += v (compartilhado)   | ✗ FALHOU  | 27%       |

JCS3 e JCS4 falham **como esperado** — eles testam a race condition deliberadamente. A implementação real utiliza arrays locais, tornando-os seguros.

| Abordagem   | B1 (s/op) | B2 (s/op) | $\pm$ B1 | $\pm$ B2 |
|-------------|-----------|-----------|----------|----------|
| Serial      | 212.57    | 60.05     | 34.48    | 2.46     |
| v2-platform | 214.87    | 60.69     | 29.10    | 6.41     |
| v2-virtual  | 211.64    | 71.61     | 260.68   | 227.05   |
| v2-hybrid   | 215.69    | 60.44     | 59.78    | 5.33     |

Nenhuma melhoria em relação ao serial. O parsing de CSV ainda é o gargalo para todas as variantes. v2-virtual apresenta variância extrema ( $\pm 260$  s) — ForkJoinPool não é projetado para tarefas CPU-bound que nunca bloqueiam.

## v2 — Throughput JMeter (ParallelGC)

| Threads | v2-platform | v2-virtual | v2-hybrid |
|---------|-------------|------------|-----------|
| 1       | 1.65        | 1.62       | 1.60      |
| 2       | 2.11        | 2.13       | 2.14      |
| 4       | 2.62        | 2.14       | 2.71      |
| 8       | 2.79        | 3.77       | 3.38      |
| 16      | 4.39        | 4.34       | 4.40      |

exec/min, threads JMeter  $\times$  abordagem

Todas as variantes atingem  $\approx 4,5$  exec/min com 16 threads. Com 1 thread, apenas 1,6 exec/min — cada execução leva  $\approx 37$  s para concluir.

| GC         | Serial | v2-platform | v2-virtual | v2-hybrid |
|------------|--------|-------------|------------|-----------|
| G1GC       | 1.56   | 1.65        | 1.59       | 1.60      |
| ZGC        | 1.50   | 1.42        | 1.55       | 1.45      |
| ParallelGC | 1.52   | 1.59        | 1.62       | 1.59      |

exec/min — baixa concorrência

Com 1 thread, os três GCs ficam dentro de **0,23 exec/min** entre si. A pressão no heap é mínima — o GC mal é ativado e não tem impacto mensurável no throughput.



## v2 — Comparação de GC: 16 Threads JMeter

| GC         | Serial | v2-platform | v2-virtual | v2-hybrid |
|------------|--------|-------------|------------|-----------|
| G1GC       | 4.66   | 4.49        | 4.55       | 4.53      |
| ZGC        | 4.40   | 4.10        | 4.09       | 4.09      |
| ParallelGC | 4.85   | 4.39        | 4.34       | 4.40      |

exec/min — alta concorrência

- **ZGC** é consistentemente o mais baixo — write barriers adicionam custo mesmo com um heap pequeno ( $\approx 250$  MB)
- **G1GC** e **ParallelGC** ficam dentro de 0,5 exec/min entre si em todas as variantes
- O impacto geral do GC permanece **pequeno** ( $< 10\%$ ) — as pausas somam  $< 1\%$  do tempo total de execução

### v2-platform

- 86,6% parsing de CSV (main)
- 13,4% algoritmo (workers)

### v2-virtual

- 99,8% parsing de CSV (main)
- 0,2% algoritmo (workers)

### v2-hybrid

- 0% CSV visível
- 99,9% algoritmo (workers)

O coordenador hybrid é desmontado durante readLine — o parsing de CSV torna-se invisível ao profiler de CPU. Isso revelou a percepção real: **o problema não é a coordenação, é a própria leitura.**

## Implementação Otimizada (v3)

---

## v3 — A Correção: Carregar Uma Vez, Computar Muitas Vezes

**Mudança-chave:** carrega todos os 5,75 milhões de pontos em `double[] []` *antes* do loop de iteração. O CSV é lido exatamente uma vez.

| Componente      | Mudança                                                               |
|-----------------|-----------------------------------------------------------------------|
| CsvDataset      | novo <code>loadAllPoints()</code> → <code>double[] []</code>          |
| KMeans          | nova assinatura <code>cluster(double[] [], ...)</code>                |
| KMeans          | <code>CountDownLatch(N)</code> em vez de lista de <code>Future</code> |
| KMeans          | <code>N</code> chunks fixos em vez de lotes <code>BATCH_SIZE</code>   |
| KMeansBenchmark | pontos pré-carregados em <code>@Setup(Level.Trial)</code>             |
| KMeansSampler   | <code>static volatile sharedPoints</code> (double-checked locki       |

Mesmos quatro testes, mesmo resultado.

| ID   | Operação                                       | Resultado | Freq. máx |
|------|------------------------------------------------|-----------|-----------|
| JCS1 | <code>closestCentroid</code> (somente leitura) | ✓ PASSOU  | —         |
| JCS2 | <code>squaredDistance</code> (somente leitura) | ✓ PASSOU  | —         |
| JCS3 | <code>counts[c]++</code> (compartilhado)       | ✗ FALHOU  | 4,05%     |
| JCS4 | <code>sums[c][d] += v</code> (compartilhado)   | ✗ FALHOU  | 5,95%     |

O design de acumulação local é preservado na v3 (`processChunk` usa o mesmo padrão que `processAssignBatch`). JCS1 e JCS2 ainda passam.

| ID | Benchmark                | Serial | v3           |
|----|--------------------------|--------|--------------|
| B1 | Execução completa (s/op) | 212.57 | 4.55 ± 17.98 |
| B2 | Por iteração (s/op)      | 60.05  | 1.60 ± 0.17  |
| B3 | closestCentroid (ns/op)  | 96.82  | 96.82        |
| B4 | squaredDistance (ns/op)  | 9.31   | 9.31         |

B2: 60,05 → 1,60 s/op — **37× mais rápido por iteração.**

B3 e B4 são inalterados: o hotspot aritmético é o mesmo, agora ele domina.

## v3 — Throughput JMeter (ParallelGC)

| Threads | exec/min | Latência média (ms) | Erros |
|---------|----------|---------------------|-------|
| 1       | 52.5     | 1,145               | 0     |
| 2       | 56.0     | 2,010               | 0     |
| 4       | 57.1     | 3,796               | 0     |
| 8       | 57.2     | 7,457               | 0     |
| 16      | 62.3     | 13,374              | 0     |

O throughput sobe de 52,5 para 62,3 exec/min (1 → 16 threads).

O ganho modesto é esperado: cada execução já satura todos os núcleos de CPU internamente.

### I/O

- 0 eventos `jdk.FileRead`
- CSV lido apenas na inicialização
- Todas as iterações: somente memória

### GC

- 60 eventos (vs 1.000+ na v2)
- 1.024 ms total de pausas
- Heap máximo:  $\approx 2,7$  GB

### O que domina agora?

`KMeans.squaredDistance` e `closestCentroid` dominam o perfil — o custo computacional real, finalmente visível.

### Trade-off:

$\approx 250$  MB heap (v2)  $\rightarrow \approx 2,7$  GB (v3)



## Resultados e Lições Aprendidas

---

# Comparação Geral

| Abordagem   | B2 (s/op) | 1t (exec/min) | 16t (exec/min) | Heap    |
|-------------|-----------|---------------|----------------|---------|
| Serial      | 60.05     | —             | —              | ≈250 MB |
| v2-platform | 60.69     | 1.65          | 4.49           | ≈250 MB |
| v2-virtual  | 71.61     | 1.62          | 4.55           | ≈250 MB |
| v2-hybrid   | 60.44     | 1.60          | 4.53           | ≈250 MB |
| v3          | 1.60      | 52.5          | 62.3           | ≈2.7 GB |

B2 = custo por iteração (JMH)    throughput = JMeter ParallelGC

As variantes v2 são equivalentes ao serial. A v3 atinge **37×** de speedup por iteração e **32×** de ganho de throughput (1 thread).

### 1. **Perfile antes de paralelizar.**

87% do tempo era parsing de CSV. Paralelizar os 13% (computação) primeiro produziu zero de ganho mensurável.

### 2. **Gargalos migram.**

Uma vez que o I/O foi eliminado, `squaredDistance` tornou-se o hotspot visível — o próximo alvo de otimização.

### 3. **Threads virtual não são para tarefas CPU-bound.**

Elas brilham ao bloquear em I/O. Em computação pura, adicionam variância de escalonamento sem benefício de desempenho.

### 4. **A acumulação local escala de forma limpa.**

Sem locks, sem atomics, sem `synchronized`. Cada worker escreve em arrays privados; o coordenador reduz uma vez. O JCStress verificou a corretude formalmente.

### 5. **Memória é um trade-off.**

Carregar 1,75 GB antecipadamente reduziu os eventos de GC em  $16\times$  mas torna a abordagem inviável para datasets muito grandes.

Obrigado!